

TP2

Projet analytique en groupe

Projet en trinômes avec thème libre, et oral à la séance suivante.

MATIÈRE	SQL avancé (PostgreSQL)
ANNÉE SCOLAIRE	2026-2027
INTERVENANT	Hédi Boulord
GROUPE CIBLE	CYBERSEC B2, Ynov Campus Montpellier
SÉANCE	2
DERNIÈRE MISE À JOUR	09/10/2026

TP2 : Votre projet analytique en groupe

Durée : 4h en séance, finalisation possible avant la prochaine séance. Travail en trinômes (groupes de 3, ou 2 si le nombre d'élèves l'impose). Présentation orale : prochaine séance (mardi 13/10/2026).

Le principe

Votre groupe **invente un thème**, construit sa propre base de données, puis écrit des requêtes analytiques dessus en utilisant les notions du cours (fonctions de fenêtre, CTE, récursivité, agrégations avancées). La semaine suivante, chaque groupe présente son travail à l'oral devant la classe.

Le but : montrer que vous savez transformer une question métier en requête SQL avancée, et expliquer ce que vous avez écrit.

Déroulé conseillé

Durée	Étape
0h00 à 1h00	Lecture du cours écrit et test des exemples (chargez <code>seance2-donnees.sql</code>)
1h00 à 1h20	Formation des groupes, choix du thème, validation avec l'intervenant
1h20 à 1h55	Palier 1 : conception et données
1h55 à 2h40	Palier 2 : fonctions de fenêtre
2h40 à 3h20	Palier 3 : CTE et récursivité
3h20 à 3h50	Palier 4 : rapport d'agrégation
3h50 à 4h00	Préparation de l'oral, dépôt des fichiers

Personne n'est censé tout finir en séance : le palier 4 et le bonus peuvent se terminer avant l'oral. **Mieux vaut des paliers 1 à 3 solides et compris qu'un palier 5 bâclé.**

Travailler avec l'IA

Un assistant IA est autorisé pour avancer. Mais à l'oral, **chaque membre du groupe doit savoir expliquer n'importe quelle requête** du projet, ligne par ligne. Une requête que vous ne comprenez pas se voit tout de suite. Relisez et testez tout ce que l'IA vous propose.

Le projet de groupe

Choisir son thème

Le thème doit contenir des **événements qui se répètent dans le temps** (des ventes, des parties, des visionnages, des trajets...), parce que c'est ce qui rend les calculs

analytiques intéressants. Voici des pistes, ou proposez les vôtres :

- Une plateforme de streaming (abonnés, contenus, catégories, visionnages)
- Une salle de sport (adhérents, cours, coachs, réservations)
- Un jeu en ligne (joueurs, parties, classes de personnages, achats)
- Une plateforme de covoiturage (conducteurs, passagers, trajets, villes)
- Une chaîne de restaurants ou de livraison (commandes, plats, restaurants, livreurs)
- Une banque (clients, comptes, virements, agences)
- Un festival ou une billetterie (événements, lieux, billets, spectateurs)
- Une boutique en ligne autre que NovaShop (sneakers, jeux vidéo, plantes...)

Validez votre thème avec l'intervenant avant de commencer à coder (5 minutes par groupe). Le thème doit permettre de remplir tous les paliers ci-dessous.

Organisation du dépôt de travail

Créez un dossier (ou un dépôt GitHub public) nommé `projet-sql-nomdugroupe` avec cette structure :

```
projet-sql-nomdugroupe/  
├─ README.md      (thème, schéma, comment charger la base)  
├─ schema.sql     (création des tables)  
├─ seed.sql       (insertion des données)  
├─ requetes.sql   (toutes vos requêtes analytiques, commentées)  
└─ JOURNAL.md     (choix faits, difficultés rencontrées)
```

Chaque requête de `requetes.sql` est précédée d'un commentaire qui donne **la question métier** posée, par exemple :

```
-- Q3 : Quel est le top 3 des contenus les plus regardés dans chaque catégorie ?  
WITH classement AS ( ... )  
SELECT ...
```

Si vous utilisez GitHub, faites des **commits réguliers** avec des messages clairs : l'historique est regardé.

Palier 1 : Conception et données (obligatoire)

Ce qu'on attend :

1. **Au moins 4 tables** reliées par des clés étrangères, avec clés primaires, `NOT NULL`, et au moins deux contraintes `CHECK` ou `UNIQUE` pertinentes.
2. **Une table hiérarchique** : une table qui pointe vers elle-même (`parent_id`, `manager_id` ...), par exemple des catégories avec sous-catégories, ou des employés avec un manager. Elle servira au palier 3.
3. **Une table d'événements** (ventes, visionnages, parties...) avec une **date** et une **valeur numérique** (montant, durée, score...).

4. **Au moins 100 lignes** dans la table d'événements, réparties sur **au moins 3 mois**, et au moins 15 lignes dans chacune des autres tables principales.
5. Un schéma de votre base (dessin, tableau, ou texte dans le `README.md`).

Astuce pour remplir beaucoup de lignes : ne tapez pas 100 `INSERT` à la main. Générez-les avec `generate_series` et `random()`. Exemple (150 visionnages entre juillet et septembre) :

```
SELECT setseed(0.42); -- pour obtenir toujours les mêmes données
INSERT INTO visionnages (abonne_id, contenu_id, date_vision, minutes_vues)
SELECT 1 + floor(random() * 20)::int,          -- un des 20 abonnés
       1 + floor(random() * 15)::int,          -- un des 15 contenus
       DATE '2026-07-01' + floor(random() * 90)::int, -- une date sur 90 jours
       5 + floor(random() * 100)::int          -- entre 5 et 104 minutes
FROM generate_series(1, 150);
```

Les tables parentes (abonnés, contenus...) doivent être remplies **avant**. Pour des noms variés, utilisez un tableau : `(ARRAY['gratuit', 'standard', 'premium'])[1 + (g % 3)]`.

Vérification : `SELECT COUNT(*), MIN(date), MAX(date) FROM votre_table_evenements;` doit montrer au moins 100 lignes et 3 mois d'écart.

Palier 2 : Fonctions de fenêtre (obligatoire)

Écrivez au moins **6 requêtes** qui répondent à des questions métier, chacune utilisant une fonction de fenêtre **différente ou un usage différent**. Il faut couvrir au minimum :

1. Un **classement** avec `RANK` ou `DENSE_RANK` (avec `PARTITION BY`).
2. Un **top N par groupe** (par exemple le top 3 de chaque catégorie), avec une CTE.
3. Un **cumul** (`SUM()` OVER (`ORDER BY ...`)), par exemple le chiffre d'affaires cumulé par mois.
4. Une **évolution d'une période à l'autre** avec `LAG` ou `LEAD`, exprimée en pourcentage.
5. Une **part du total** ou un **écart à la moyenne du groupe** (`SUM()` OVER (`...`) ou `AVG()` OVER (`PARTITION BY ...`)).
6. Une **moyenne mobile** avec un cadre explicite (`ROWS BETWEEN ...`) **ou** un découpage en tranches avec `NTILE`.

Pour chaque requête, notez dans `requetes.sql` la question métier, et dans `JOURNAL.md` ce qu'elle apprend sur vos données (une phrase suffit).

Palier 3 : CTE et récursivité (obligatoire)

1. **Deux requêtes** utilisant chacune au moins **2 CTE enchaînées** (la deuxième s'appuie sur la première). Exemple : calculer un total par client, puis garder ceux au-

dessus de la moyenne.

2. **Une requête avec** `WITH RECURSIVE` sur votre table hiérarchique : l'arbre complet avec niveau et chemin, ou le total d'une branche (le chiffre d'affaires d'une catégorie avec toutes ses sous-catégories, par exemple).
3. **Une requête qui fait apparaître les jours (ou les mois) sans activité**, avec `generate_series` et un `LEFT JOIN`.

Palier 4 : Rapport d'agrégation (obligatoire pour valider le projet)

1. Un **tableau croisé** avec `FILTER` : une colonne par mois, ou par catégorie, ou par type de client.
2. Un **rapport avec sous-totaux** : `ROLLUP`, `CUBE` ou `GROUPING SETS`, avec des libellés propres (`GROUPING()` ou `COALESCE`) à la place des `NULL`.
3. Une requête qui compare la **moyenne et la médiane** de votre valeur numérique (`percentile_cont`), avec une phrase d'interprétation dans le journal : quelle valeur représente le mieux vos données, et pourquoi ?
4. Une requête « **tableau de bord direction** » de votre choix, qui combine au moins deux techniques du cours en un seul résultat lisible.

Palier 5 : Défis bonus (facultatif)

Choisissez au moins un défi, ou proposez le vôtre :

- **Une vue** : transformez votre meilleur rapport en `CREATE VIEW`, et interrogez-la.
- **Un index et EXPLAIN** : comparez le plan d'exécution (`EXPLAIN ANALYZE`) d'une requête **avant et après** la création d'un index, avec un volume plus grand (par exemple 100 000 lignes générées). Expliquez ce que vous observez.
- **Détection de séries** : trouvez, avec `LAG` et un cumul, les périodes de croissance ou de baisse de plusieurs mois d'affilée.
- **Un problème au choix** de votre thème (utilisateurs inactifs depuis 30 jours, premier achat de chaque client, rétention) résolu avec les fonctions de la séance.

L'oral (prochaine séance)

Format : 10 minutes de passage par groupe, 5 minutes de questions.

1. **Le thème et le schéma** (2 minutes) : présentez la base, ses tables, la table hiérarchique.
2. **La démonstration** (5 minutes) : exécutez en direct **3 requêtes de votre choix**, dont au moins une fonction de fenêtre et une CTE. Pour chacune : la question métier, le principe de la requête, le résultat obtenu.
3. **Le retour d'expérience** (2 minutes) : une difficulté rencontrée et comment vous l'avez résolue, ce que vous referiez autrement.

4. **Les questions** (5 minutes) : l'intervenant désigne **un membre au hasard** pour expliquer une requête qu'il n'a pas présentée. Tout le monde doit donc comprendre tout le projet.

Préparez votre base de façon à pouvoir la recharger en une minute (`schema.sql` puis `seed.sql`) et ayez vos requêtes prêtes dans un fichier. Testez votre démo sur l'ordinateur qui servira à la présentation.

Grille d'évaluation

Critère	Poids
Palier 1 : conception et données	15 %
Palier 2 : fonctions de fenêtre	25 %
Palier 3 : CTE et récursivité	20 %
Palier 4 : rapport d'agrégation	15 %
Qualité du travail rendu (requêtes commentées, README, journal, organisation)	10 %
Oral (clarté, démonstration, réponses aux questions)	15 %
Défis bonus du palier 5	jusqu'à 5 % supplémentaires

La note de groupe peut être ajustée de quelques points pour un membre qui n'a visiblement pas compris le projet à l'oral.

À rendre

Avant le début de la prochaine séance, déposez votre dossier `projet-sql-nomdugroupe` (zip ou lien du dépôt) sur Moodle, avec : `README.md` , `schema.sql` , `seed.sql` , `requetes.sql` , `JOURNAL.md` .

En cas de blocage

Si vous bloquez plus de 5 minutes sans avancer en groupe, appelez l'intervenant. Les erreurs les plus fréquentes sont listées à la fin du cours (partie 10) : relisez-les avant de demander.